

# Neural Networks Executive Summary

## 1. Abstract

This report outlines the design, training, and evaluation of an Enhanced Multi-Layer Perceptron (ANN) and Enhanced Convolutional Neural Network (CNN) for Fashion-MNIST classification—a 70,000-image grayscale dataset (60k train/10k test) of 28×28 clothing items across 10 classes (e.g., T-shirt/top, trouser, shirt, sneaker). Using PyTorch, the focus was preprocessing, model comparison, and innovations like augmentation/regularization to assess spatial vs. flattened processing. The CNN achieved 88.72% test accuracy vs. ANN's 79.79%, highlighting conv layers' superiority, with analysis of confusions (e.g., shirt vs. coat) and overfitting mitigation.

## 2. About Data

Fashion-MNIST was loaded via torchvision, split into 50k train (augmented), 10k val (plain), and 10k test. Normalization used dataset mean (0.1307)/std (0.3081) for stability. Augmentation (train-only): RandomRotation (10°), RandomHorizontalFlip(p=0.5), RandomCrop(28, padding=2). DataLoaders: batch\_size=128, shuffle=True (train), num\_workers=2. This supports efficient gradient optimization, with random\_split for subsets.

## 3. AI Models Building

### ANN (Enhanced MLP)

Flattened input (784) → hidden (512→256→128, ReLU) → output (10). Added BatchNorm1d (post-linear) and Dropout(p=0.5) to curb overfitting. Params: 569,226. Forward: Flatten → ReLU-BN-Dropout sequence.

### CNN (Enhanced ConvNet)

Three blocks: Conv2d (1→32/64/128, 3×3, pad=1) → BatchNorm2d → ReLU → MaxPool (2×2) → Dropout2d(p=0.25); flatten (1152) → FC (256, ReLU-BN-Dropout p=0.5) → output. Params: 391,370. Captures hierarchies; size calc ensures correct flattening.

## 4. Model Training and Evaluation

### Training Setup

Reusable train\_model: CrossEntropyLoss, RMSprop (lr=0.01, alpha=0.9, wd=1e-4), 20 epochs. ReduceLROnPlateau (factor=0.5, patience=3) on val loss. Per epoch: Train (forward/backprop), val (eval mode). Save/load

best by val acc. Logs: ANN ~73% train/80% val; CNN ~85% train/89% val.

## 5. Validation & Test Performance

Val monitored overfitting (e.g., CNN peak 89.29%). Test via evaluate\_model: Acc, confusion matrix (Seaborn heatmap), per-class recall.

### Model Test Acc

ANN 79.79%

CNN 88.72%

Curves show CNN's faster convergence/less divergence. Per-class:

| Class      | ANN (%) | CNN (%) |
|------------|---------|---------|
| T-shirt    | 77.3    | 83.1    |
| Trouser    | 96.2    | 97.4    |
| Pullover   | 60.4    | 80.6    |
| Dress      | 88.9    | 91.1    |
| Coat       | 72.1    | 86.1    |
| Sandal     | 82.7    | 93.1    |
| Shirt      | 43.9    | 67.1    |
| Sneaker    | 88.5    | 98.0    |
| Bag        | 93.3    | 97.0    |
| Ankle boot | 94.6    | 93.7    |

CNN lifts ambiguous classes (e.g., Shirt +23%).

## 6. Confusion Matrix Observations

Diagonals peak for distinct items (Sneaker 98%, Trouser 97%). Errors in uppers: ANN Shirt→T-shirt (233), Pullover→Coat (185); CNN Shirt→T-shirt (107), Pullover→Coat (81). Grayscale similarity drives >50-instance off-diags (e.g., Sandal→Sneaker: 97 ANN, 61 CNN).

## 7. Conclusion

CNNs excel for images (88.72% vs. 79.79%), leveraging convs for spatial feats absent in ANN. Augmentation, RMSprop, BatchNorm/Dropout innovations reduced overfitting (~9% gain). Confusions reveal shape ambiguities; metrics (acc, matrices, curves) expose strengths (CNN textures) and gaps (uppers).